

NADA GURUKULAM

ERP + LMS

Institutional Architecture & Development Manual

Master blueprint for designing and developing the Nada Gurukulam web application

Version 1.0 | Architecture Foundation

How to Use This Manual

Use this document as the master reference before database design and coding. When a new feature is requested, first identify the relevant domain, entity, relationship, permission and workflow. Avoid creating a new table or field simply because a new screen is needed.

The recommended order is: Data Dictionary → Entity Relationship Model → Permissions → Workflows → Database Schema → APIs → UI → Testing → Deployment.

1. Purpose of this Manual

- This manual is the master blueprint for designing and developing a professional Nada Gurukulam ERP + LMS web application. It is intended to be usable by an institution, product owner, system architect, database designer, UI/UX designer, and developer.
- The system should be designed as one institutional information system with shared master data and separate functional domains. ERP functions manage institutional operations; LMS functions manage teaching and learning. Both must use the same identity, student, academic, and organisational records.
- The architecture must be configurable rather than hard-coded so that UG, PG, Diploma, certificate, short-term and future programmes can coexist without redesigning the platform.

2. Core Design Principles

- Single source of truth: each important fact is stored once and referenced by other modules.
- Separate master data from transactional data. Programmes, courses, roles and rooms are master definitions; enrolments, attendance, submissions and payments are transactions.
- Separate designation from system role. A person's institutional designation does not automatically determine their software permissions.
- Separate academic definitions from academic instances. A Course is a definition; a Course Offering is the instance taught to a cohort in a particular term.
- Use stable IDs for people, students, programmes, courses and transactions. Never use names as primary keys.
- Preserve historical data. Do not overwrite curriculum, grades, enrolments or employment records that have already been used.
- Use effective dates and status fields where relationships can change over time.
- Design for least-privilege access, auditability, privacy, backup and recoverability from the beginning.
- Keep business rules configurable wherever possible: grading schemes, semester structure, attendance rules, assessment weights, fee heads, programme duration and permissions.

3. System Domains

- Organisation and governance; Identity and Access; People; Admissions; Academic Management; Student Lifecycle; LMS; Attendance; Assessment and Examination; Residential Life; Staff/HR; Events and Performances; Finance and Fees; Documents and Media; Communication; Reporting and Analytics; Audit and Compliance.

4. Master Architecture

- Top-level relationship: Institution → Organisation; Institution → Academic Structure; Person → User Account; Person → Student/Employee/Other Role; Student → Enrolment → Programme/Stream/Curriculum; Enrolment →

Course Registration → Course Offering → Course; Course → Module → Lesson → Learning Activity; Course Offering → Faculty Assignment; Student → Attendance/Assessment/Result/Residential/Performance records.

- Recommended separation: organisation answers 'where does this person/unit belong?'; identity answers 'who is this person?'; role answers 'what may they do?'; academics answers 'what are they studying/teaching?'; LMS answers 'how is learning delivered?'; ERP answers 'what institutional transactions occur?'.

5. Organisation Model

- Entities: Institution, Campus, School/Faculty, Department, Centre, Unit, Office, Position, Reporting Relationship.
- Recommended hierarchy: Institution → Campus → School/Faculty → Department/Centre/Unit. Do not force every operational group to be a Department.
- Example: Nada Gurukulam → Academic → Music → Carnatic Vocal Unit; Nada Gurukulam → Residential Life → Girls Hostel; Nada Gurukulam → Director's Office.
- Reporting relationships should be stored independently so that a person can report to another person without making the organisational tree rigid.

6. People, Identity and Access

- Person is the human record. User Account is the login identity. Employee, Student, Parent/Guardian, Faculty, Guru, Mentor, Warden and Visiting Artist are relationships or profiles attached to a Person.
- Recommended flow: Person → User Account (optional) → Roles → Permissions. A person may have multiple roles and a role may have multiple permissions.
- Use scope-based access: institutional, campus, department, course, assigned-student, self, or child-of-parent. Example: faculty can edit attendance only for assigned course offerings; HOD can review a department; Director can view institution-wide reports.
- Important distinction: Designation is an HR/institutional attribute; Role is an application access concept.

7. Academic Master Structure

- Programme Type: UG, PG, Diploma, Certificate, Short-Term, Foundation, etc.
- Programme: BPA, MPA, Diploma in Indian Classical Music, etc.
- Stream/Specialisation: Carnatic Vocal, Hindustani Vocal, Bharatanatyam, Mridangam, Tabla, Carnatic Violin, Flute, Veena, etc.
- Curriculum Version: an immutable version of the programme curriculum used by a cohort.
- Academic Year: e.g. 2026–27. Programme Year: Year 1, Year 2, etc. Semester/Term: Semester 1, Semester 2, or another configurable term type.
- Batch/Cohort: the group admitted together, e.g. BPA Bharatanatyam 2026–2030.
- Course: reusable academic definition. Course Offering: an actual scheduled instance of that course for a term/cohort, with faculty, students, timetable and assessments.
- Module: a logical section of a course. Lesson: an individual teaching unit. Learning Outcome: measurable expected competency.

8. Curriculum Versioning

- Never modify an already-used curriculum in place. Create a new Curriculum Version when requirements change.
- Example: BPA Bharatanatyam → Curriculum 2026; BPA Bharatanatyam → Curriculum 2028. Students admitted under 2026 continue to be governed by that version unless an approved migration rule exists.

- Curriculum should map courses to programme year/semester, credits or workload, prerequisites, learning outcomes, assessment scheme and course type.

9. Student Lifecycle

- Recommended workflow: Application → Screening → Audition/Entrance → Selection → Admission → Enrolment → Batch Allocation → Course Registration → Classes → Attendance → Assessment → Results → Progression → Graduation → Alumni.
- Student Status should be configurable: Applicant, Admitted, Active, Leave, Suspended, Withdrawn, Discontinued, Graduated, Alumni, etc.
- Enrolment is the student's relationship with a programme. It should contain programme, stream, curriculum version, batch, admission date, academic status and effective dates.

10. Course Registration and Faculty Assignment

- Do not connect Student directly to a generic Course for operational records. Use Student → Course Registration → Course Offering → Course.
- Do not permanently attach Faculty to Course. Use Course Offering → Faculty Assignment → Person/Employee. This supports multiple teachers, guest gurus, accompanists and team teaching.
- Course Offering can contain academic year, term, section, timetable, room, capacity, status and teaching team.

11. Guru, Mentor and Advisor Relationships

- Make mentoring explicit. Student may have a Guru, Academic Mentor, Course Faculty, Hostel Mentor and other advisors. These are separate relationship types.
- Use a Student Relationship table with relationship type, person, start date, end date, scope and status. This preserves history when mentors change.
- Guru-Shishya relationships can support progress notes, mentor assessments and student development records without being confused with course teaching assignments.

12. LMS Architecture

- Recommended learning hierarchy: Course → Module → Lesson → Learning Resources and Activities.
- Resources can include video, audio, PDF, notation, image, external link and downloadable material. Activities can include assignment, quiz, discussion, practice submission, reflection and project.
- Every LMS item should have ownership, visibility, publication status, ordering, version, created/updated timestamps and access rules.
- Learning should connect outcomes to activities and assessment: Learning Outcome → Content/Activity → Evidence/Submission → Evaluation → Feedback → Competency/Result.

13. Attendance

- Attendance should be based on actual Class Sessions rather than a single percentage field.
- Relationship: Course Offering → Class Session → Attendance Record → Student.
- Class Session should contain date/time, teacher, room/online mode, topic, session type and status. Attendance should support Present, Absent, Excused, Late and other configurable states.

- Attendance summaries are derived data; the session-level records remain the source of truth.

14. Assessment and Examination

- Use a generic assessment engine that can support academic and artistic evaluation.
- Structure: Course Offering → Assessment Plan → Assessment → Submission/Performance → Evaluation → Grade/Score → Result.
- Assessment types may include theory, practical, viva, assignment, project, recital, internal assessment, semester examination and competency evaluation.
- Use configurable rubrics. For Bharatanatyam: technique, rhythm, posture, expression, coordination, memory, stage presence. For vocal: shruti, laya, raga understanding, sahitya, manodharma, presentation. Rubrics should be data-driven, not coded into the application.

15. Grading and Results

- Keep raw marks/scores, maximum marks, weightage, grade, grade point, evaluator and feedback separately where appropriate.
- Do not overwrite results. Corrections should create an auditable change record.
- Support configurable grading schemes and passing rules. Final transcript values should be derived from approved result records and curriculum rules.

16. Residential / Hostel ERP

- Model: Campus → Hostel → Block → Floor → Room → Bed → Allocation → Student.
- Related records: Warden Assignment, Hostel Mentor, Leave Request, Outing, Return, Room Transfer, Incident/discipline record where institutionally appropriate.
- Room and bed allocation must be time-bound so historical occupancy can be retained.
- Residential records should be permission-restricted and separated from general academic visibility.

17. Staff and HR

- Employee profile should include designation, department/unit, employment type, joining date, qualifications, expertise, reporting manager and status.
- Operational records may include workload, faculty assignment, attendance, leave, professional development and performance review.
- HR data should have stricter access than ordinary academic records.

18. Events, Productions and Performances

- Model Event → Production/Programme → Performance → Participant/Artist → Rehearsal/Travel/Logistics.
- Student participation should be a separate relationship so an event can include students, faculty, gurus, guest artists and external artists.
- This domain can generate an artistic portfolio for each student: performances, productions, workshops, tours and roles.

19. Documents and Media

- Use a central document/media service. A document has type, owner entity, storage reference, access classification, version, checksum where needed, and retention status.
- Do not scatter file paths across application tables. Store metadata in the database and binary/media objects in controlled storage.
- Support document categories for admission, identity, academic, HR, certificates, agreements, LMS resources and institutional records.

20. Communication and Notifications

- Use one notification engine for email, in-app, SMS or WhatsApp integrations.
- Notifications should be generated from domain events such as assignment deadlines, result publication, leave approval, fee due dates, timetable changes or announcements.
- Store delivery status and timestamps for auditability.

21. Reporting and Analytics

- Build dashboards from reporting views or an analytics layer rather than embedding complex reporting logic inside every module.
- Core dashboards: Director, Academic, HOD, Faculty, Mentor, Student, Parent, Hostel, Finance, HR and Institutional Analytics.
- Examples: attendance trends, student progression, course completion, assessment performance, hostel occupancy, faculty workload, programme intake, performance participation and institutional KPIs.

22. Student 360°

- A Student 360 should combine academic, residential and artistic information while respecting permissions.
- Sections: identity, enrolment, programme/stream, curriculum, attendance, assessments, results, mentor/guru relationships, hostel, leave, discipline if authorised, performances, workshops, achievements, documents and communication history.
- Student 360 is a view assembled from source modules; it should not duplicate all underlying data.

23. Master vs Transactional Data

- Master data: Institution, Campus, Department, Programme, Stream, Curriculum, Course, Academic Year, Term, Room, Role, Permission, Assessment Template.
- Transactional data: Application, Admission, Enrolment, Course Registration, Class Session, Attendance, Assignment Submission, Assessment Evaluation, Grade, Hostel Allocation, Leave, Payment, Event Participation.
- Reference/configuration data: Status lists, grading schemes, attendance states, assessment types, document types, notification templates.

24. Suggested Core Database Entities

- Organisation: institutions, campuses, organisational_units, positions, reporting_relationships.
- Identity: persons, user_accounts, roles, permissions, role_permissions, user_roles, access_scopes.

- Academic: programme_types, programmes, streams, curriculum_versions, academic_years, terms, batches, courses, curriculum_courses, course_offerings, faculty_assignments.
- Students: applicants, admissions, student_profiles, enrolments, course_registrations, progression_records, graduation_records.
- LMS: modules, lessons, resources, activities, assignments, quizzes, submissions, discussions, learning_outcomes, outcome_mappings.
- Assessment: assessment_plans, assessments, rubrics, rubric_criteria, evaluations, grades, results, examinations.
- Residential: hostels, blocks, floors, rooms, beds, allocations, leave_requests, outings, wardens.
- People/HR: employees, qualifications, expertise, workload, staff_leave, staff_attendance, performance_reviews.
- Events: events, productions, performances, participants, rehearsals, travel_records.
- Finance: fee_structures, fee_items, charges, payments, receipts, scholarships, sponsorships.
- Platform: documents, media_assets, notifications, audit_logs, announcements, system_settings.

25. Key Relationships and Cardinalities

- One Institution has many Organisational Units: 1:N.
- One Person may have zero or one User Account, depending on whether login is required: 1:0..1.
- One User Account can have many Roles and one Role can belong to many Users: N:N via user_roles.
- One Programme can have many Stream/Specialisation records where applicable: 1:N.
- One Programme can have many Curriculum Versions: 1:N.
- One Curriculum Version has many Course mappings and a Course may appear in many Curriculum Versions: N:N via curriculum_courses.
- One Course can have many Course Offerings: 1:N.
- One Course Offering can have many Students through Course Registration: N:N.
- One Course Offering can have many Faculty Assignments: 1:N.
- One Course has many Modules; one Module has many Lessons: 1:N at each level.
- One Course Offering has many Class Sessions and Assessments: 1:N.
- One Student can have many Attendance, Assessment, Residential and Performance records over time: 1:N.

26. IDs and Naming

- Use immutable system IDs, preferably UUIDs or another non-guessable identifier, plus human-readable codes.
- Examples: NG-PER-000123 for Person, NG-STU-2026-0012 for Student, NG-PRG-BPA for Programme, NG-CRS-CV101 for Course.
- Codes should be stable enough for human reference but should not be the primary database key.
- Use created_at, updated_at, created_by, updated_by for important entities. Use effective_from/effective_to for time-bound relationships.

27. Status and Lifecycle Design

- Every major workflow entity should have an explicit lifecycle. Example Application: Draft → Submitted → Under Review → Audition → Selected/Rejected → Admitted.
- Course Offering: Draft → Planned → Published → In Progress → Completed → Archived.
- Assignment: Draft → Published → Open → Closed → Evaluated.
- Enrolment: Pending → Active → Leave → Withdrawn/Discontinued → Graduated.
- Use state transitions rather than arbitrary free-text status values.

28. Security and Governance

- Use least privilege, server-side authorisation, secure password/authentication practices, session management, encryption in transit, secure secrets, input validation, rate limiting and audit logging.
- Sensitive data should have field/module-level access controls where necessary.
- Every important administrative action should record who performed it, when, what changed and, where appropriate, the previous and new values.
- Backups should be automated, tested by restoration drills, encrypted and retained according to institutional policy.

29. API and Service Architecture

- Use domain-oriented services/modules rather than one giant controller. Suggested boundaries: Identity, Organisation, Academic, Student, LMS, Assessment, Residential, HR, Events, Finance, Documents, Notifications, Reporting.
- REST or GraphQL can be used; consistency, validation and authorisation matter more than the protocol choice.
- Business rules should live in a service/domain layer rather than only in the UI.
- API responses should use stable IDs and explicit status/error structures. Avoid exposing database internals directly.

30. Frontend Architecture

- Use a common application shell with role-based navigation. The same backend can provide different dashboards for Director, HOD, Faculty, Mentor, Student, Parent and Hostel staff.
- Recommended primary navigation: Dashboard, People, Academics, Students, LMS, Assessment, Residential, Events, Finance, Reports, Administration.
- Screen visibility must be permission-driven. Do not rely only on hiding buttons; backend authorisation must enforce permissions.

31. Suggested Application Structure

- Frontend: pages/routes, components, layouts, forms, tables, dashboards, state/data layer, permission guards.
- Backend: modules/domains, controllers/API routes, services/use-cases, domain models, repositories/data access, validation, jobs/events, notification adapters.
- Shared: authentication, authorisation, audit logging, file storage, search, configuration, error handling.
- Database migrations should be version-controlled. Configuration should be environment-specific.

32. Development Phases

- Phase 0 — Architecture: data dictionary, ERD, permission model, workflows, UI wireframes.
- Phase 1 — Foundation: authentication, people, organisation, roles, permissions, audit logs.
- Phase 2 — Academic Core: programmes, streams, curriculum versions, academic years, batches, courses, course offerings.
- Phase 3 — Student Lifecycle: admissions, enrolment, course registration, student profile.
- Phase 4 — LMS: modules, lessons, resources, activities, assignments, submissions.
- Phase 5 — Attendance and Assessment: sessions, attendance, rubrics, evaluations, grades, results.
- Phase 6 — Residential and Events: hostel allocation, leave, performance participation.

- Phase 7 — Finance, HR, documents, notifications and reporting.
- Phase 8 — Advanced analytics, integrations, mobile/PWA enhancements and institutional accreditation/reporting.

33. Testing Strategy

- Unit-test business rules such as progression, grading, attendance calculations and permission checks.
- Integration-test workflows such as admission-to-enrolment and course-registration-to-LMS access.
- Permission-test every role against allowed and forbidden actions.
- Test historical data preservation, curriculum versioning, repeated courses, withdrawals, transfers and staff changes.
- Perform usability testing with actual Director's Office, administration, faculty, mentors and students before full rollout.

34. Data Migration and Implementation

- Before migration, define the source system and data owner for every field.
- Clean duplicate people and students before importing.
- Map old course names to stable course IDs rather than relying on text matching.
- Load master data first, then people, then enrolments, then transactional history.
- Validate counts, totals, grades and relationships after each migration stage.

35. Recommended End-to-End Workflow

- Application: Applicant creates profile → application submitted → documents → screening → audition/entrance → selection → approval → admission.
- Academic: admitted student → enrolment → batch/curriculum assignment → semester activation → course registration → course offering access.
- LMS: student enters course → sees published modules → accesses resources → completes activities → submits evidence → faculty evaluates → feedback → result.
- Residential: student admitted → hostel eligibility → room/bed allocation → warden assignment → leave/outing requests → occupancy reporting.
- Performance: event created → production created → students/artists assigned → rehearsals → performance → participation record → portfolio.
- Graduation: completion requirements → result approval → graduation clearance → certificate/transcript → alumni status.

36. What Should Be Configurable

- Programme types; programme duration; streams; curriculum versions; academic calendars; semester/term structures; course credits/workload; grading schemes; assessment types; rubrics; attendance states; fee structures; hostel structures; user roles; permissions; notification templates; document types; workflow statuses; dashboard widgets.
- Hard-code only genuine platform rules, not institutional policies that may change.

37. What Should Not Be Duplicated

- Do not create separate student tables for LMS, hostel and academics. Use one Student identity/profile and domain relationships.
- Do not store teacher names as text in Course Offering; reference the Person/Employee and Faculty Assignment.
- Do not store a single final attendance percentage as the only attendance data; retain session-level records.
- Do not duplicate programme names inside every student record; reference the enrolment.
- Do not overwrite old curricula or results.

38. Minimum Viable Product

- Authentication and RBAC; organisation; people; student profiles; programmes/streams; curriculum versions; academic year/term; batches; courses; course offerings; enrolment; course registration; faculty assignment; LMS modules/lessons/resources; assignments; attendance; assessment; grades; basic dashboards; audit logs.
- Hostel, finance, HR, events, advanced analytics and external integrations can follow after the academic/LMS foundation is stable.

39. Final Build Checklist

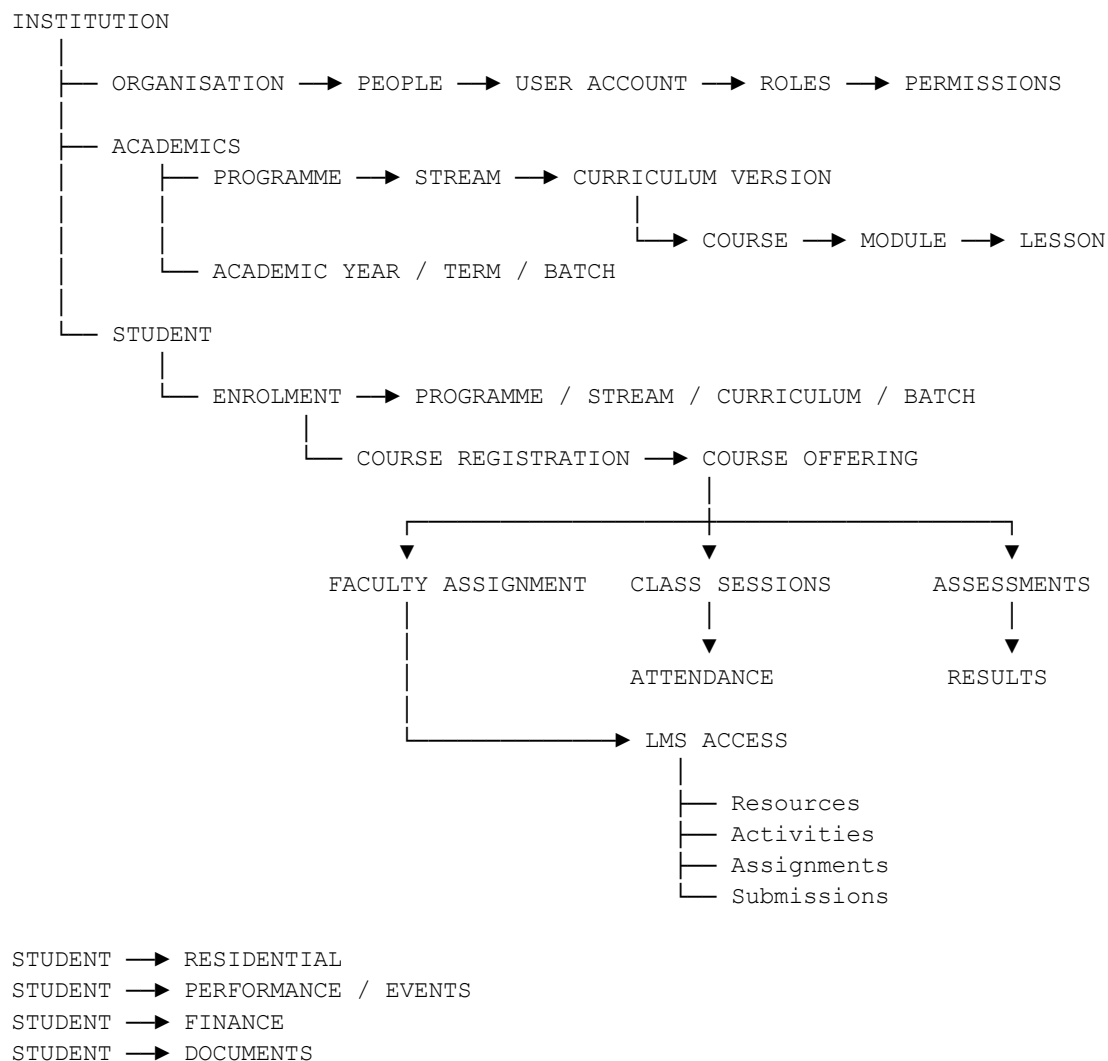
- 1. Approve institutional data dictionary.
- 2. Approve ERD and cardinalities.
- 3. Approve permission matrix.
- 4. Approve student and staff lifecycle workflows.
- 5. Define stable IDs and naming conventions.
- 6. Define curriculum versioning policy.
- 7. Define grading and assessment policies.
- 8. Define residential and performance data policies.
- 9. Design database migrations.
- 10. Build authentication/RBAC.
- 11. Build master data.
- 12. Build student lifecycle.
- 13. Build academic and LMS core.
- 14. Build attendance and assessment.
- 15. Build residential/events/finance/HR.
- 16. Build reporting.
- 17. Migrate and validate data.
- 18. Conduct role-based UAT.
- 19. Security and backup review.
- 20. Production launch with monitoring and change-control process.

40. Master Mental Model

- The most important mental model for the whole system is: Person → Identity/Role; Student → Enrolment; Enrolment → Programme/Stream/Curriculum/Batch; Curriculum → Courses; Course → Course Offering; Course Offering → Faculty + Students + Sessions + Assessments; Course → LMS Content; Student → Evidence + Results; Student → Residential/Performance/Finance relationships.

- If these relationships are designed correctly first, the screens and reports become views over a coherent institutional data model rather than separate disconnected applications.

Appendix A — High-Level Relationship Diagram



Appendix B — Recommended First Build

Start with the foundation rather than with lesson screens. The first usable vertical slice should be:

1. Create Person
2. Create User Account
3. Assign Role
4. Create Programme/Stream
5. Create Curriculum Version
6. Create Academic Year and Batch
7. Create Course and Course Offering
8. Enrol Student
9. Register Student for Course
10. Assign Faculty
11. Publish one LMS module/lesson
12. Record attendance
13. Create an assessment

14. Evaluate and publish a result

Once this vertical slice works end-to-end, the remaining modules can attach to the same foundation.

Appendix C — Architecture Rule

If a proposed feature cannot clearly answer these six questions, do not code it yet: What entity owns the data? What entity does it relate to? Is the relationship one-to-one, one-to-many or many-to-many? Who can create/read/update/delete it? What is its lifecycle? What historical information must be preserved?